Training deep networks

CIFAR-10

LECTURE 11 | GÉRON CH. 11

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Lecture 10 took the training loop out of `fit()` and gave it to you:

- backpropagation is reverse-mode automatic differentiation — one sweep for every partial derivative of a scalar loss
- tensors, `requires_grad`, and the five-line loop: `zero_grad`, forward, loss, `backward`, `step`
- two bugs that raise nothing — a missing `zero_grad()` and a missing `model.eval()` — and a metric averaged per batch
- `DataLoader`, a custom `Module`, Optuna, and a checkpoint that reloads to the same function

> **FAILURE CONDITION — A NETWORK THAT TRAINS WITHOUT LEARNING**
>
> Build a deep one — twenty hidden layers on CIFAR-10 — and train it with exactly that loop. The loss barely moves, nothing raises, nothing is NaN. Today we measure what goes wrong, derive why, and repair it.

Today

1. The failure, **instrumented** — per-layer activation and gradient statistics
2. **The mathematics** — the variance recursion, and the initialisations of Glorot and He that follow from it
3. **The method** — six repairs: initialisation, activation, normalisation, clipping, optimiser, schedule
4. The ablation: which repair actually paid

Open Lecture 11 in Colab

The specification

> **WHAT WE ARE ASKING FOR**
>
> A fully connected classifier: 3,072 inputs, **20 hidden layers** of 100 units, a logistic activation on each, and a linear head of 10 outputs. Cross-entropy loss, Adam at 0.001, batches of 128, 20 epochs.

Read that again and note what it does **not** say: nothing about how the weights start out. That is not something we are hiding — it is the ordinary case. Almost no specification mentions it.

The stack

```
def make_net(depth=20, width=100, n_in=3072, n_out=10):
    layers, prev = [], n_in
    for _ in range(depth):
        layers += [nn.Linear(prev, width), nn.Sigmoid()]
        prev = width
    layers.append(nn.Linear(prev, n_out))
    return nn.Sequential(*layers)

net = make_net()
```

Nine lines: one `nn.Sequential`, a linear layer and a logistic, with the middle repeated 20 times.

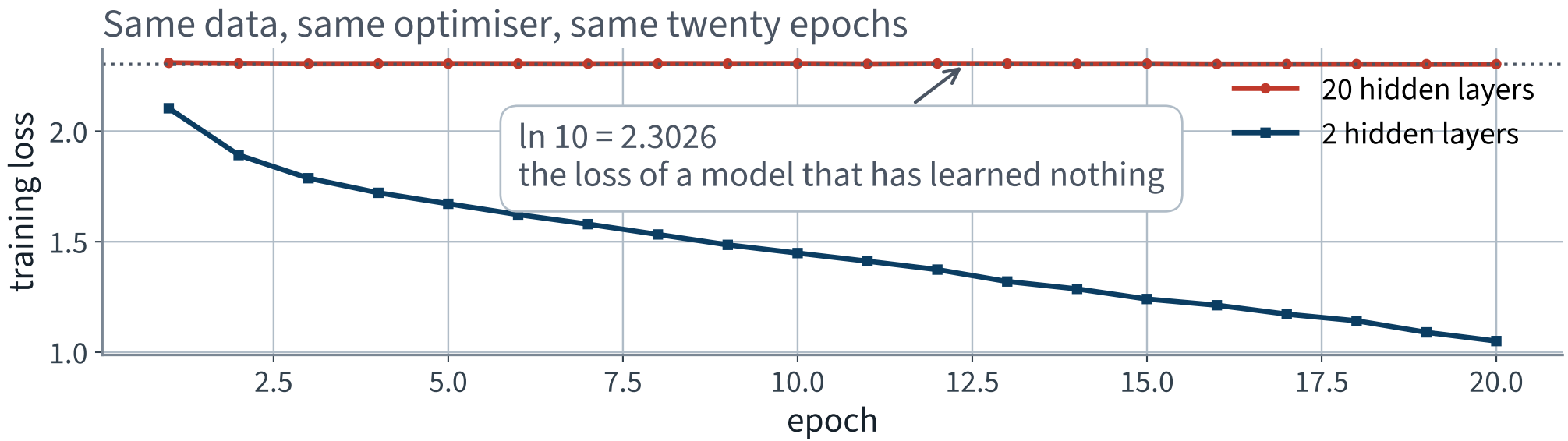
Run it

	Training loss
A model that has learned nothing, $\ln 10$	2.3026
Epoch 1	2.3091
Epoch 20	✗ 2.3036

8 seconds on mps, 20 epochs, 500,210 parameters.

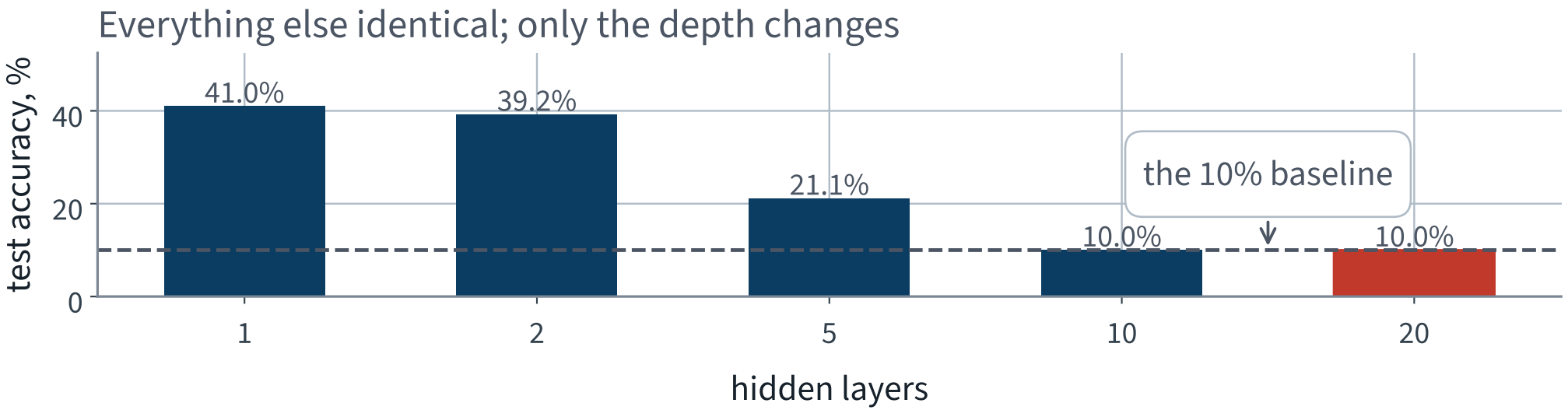
The loss moved by 0.0055 in 20 epochs.

Same data, same optimiser, same twenty epochs



✗ **The red curve is not noisy, or slow, or plateauing early. It has not started.**

Change one thing: the depth



✗ 1, 2, 5, 10 and 20 hidden layers; same data, seed, optimiser and epochs. Adding layers made it worse, and past a point made it useless. That is not what capacity is supposed to do.

What we know so far

- The loop is correct — it memorises 200 images on demand
- The data is correct — a two-layer network on the same data reaches 39.2%
- The metric is correct — counted over the set, in evaluation mode

✗ The depth is the variable, and we have no account of why

Which is where owning the training loop pays off.

What we can do now, and could not before

Backpropagation computes 500,210 partial derivatives on every batch. Under `MLPClassifier.fit()` every one of them was discarded.

They are all still here, in `p.grad`, and we can look at any of them.

- **Forward** — what does each layer output?
- **Backward** — what does each weight matrix receive?

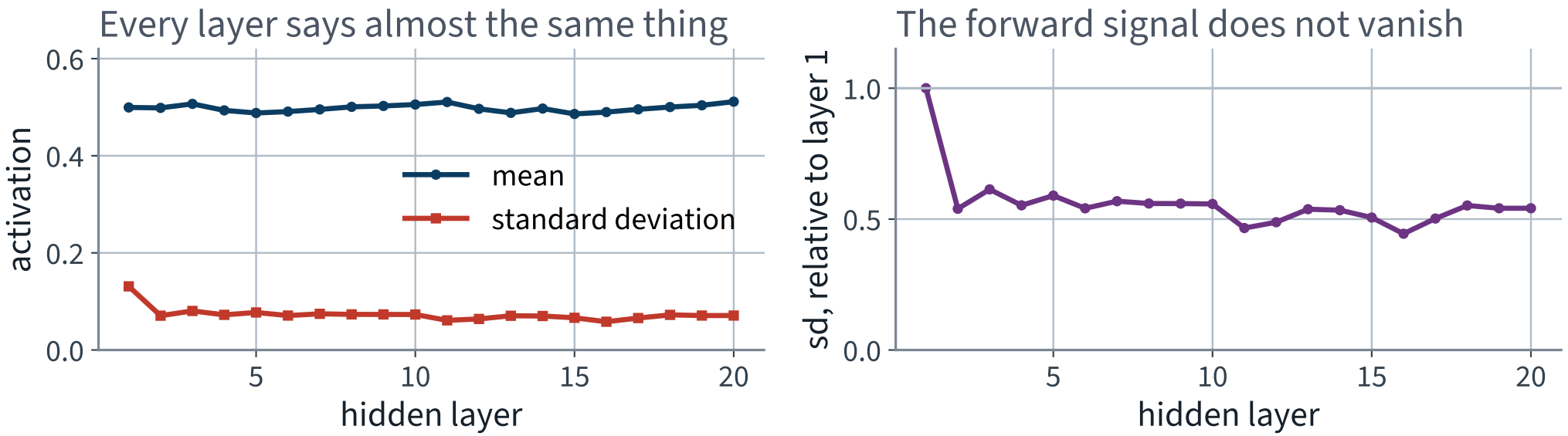
Two probes, about ten lines each. Without them the diagnosis today is “it does not train” and there is no next step.

Probe 1 · the forward pass

```
@torch.no_grad()
def activation_stats(net, X, n=512):
    net.eval()
    h, rows = X[:n], []
    for m in net:
        h = m(h)
        if isinstance(m, nn.Sigmoid):
            rows.append({"mean": float(h.mean()),
                         "sd":   float(h.std()),
                         "sat":  float(((h - .5).abs() > .45)
                                       .float().mean())})
    return rows
```

Running the modules by hand instead of calling `net(X)`. That is legal because a `Sequential` is an iterable of modules, and it is the cheapest possible instrumentation.

The probe says the forward signal is fine



Mean near a half, spread near 0.071, flat for twenty layers. **Hold that conclusion for one slide.**

The probe measured the wrong thing

`h.std()` spans the *whole tensor*, so it is dominated by the spread of the layer's random **biases across units** — which does not change with depth. It would read 0.07 in a network that had stopped computing entirely.

What carries information is the spread **down the batch** — how much a unit moves when the *input* does.

Hidden layer	1	5	10	20
`h.std()` — what we plotted	0.131	0.077	0.073	0.071
✗ `h.std(dim=0).mean()` — the signal	1.3e−01	5.9e−05	3.0e−09	2.4e−17

✗ Sixteen orders of magnitude. The forward signal does die — by a factor of 0.148 per layer, which is the number the derivation predicts from the fan-in and the same rate the gradient vanishes at. By layer 20 the network is no longer a function of its input.

✗ And probe 1's `sat` statistic was no better. It asks whether $|h - \frac{1}{2}| > 0.45$, in a band whose sd is 0.071 — a threshold 6.3 standard deviations away, and 3.4 even at layer 1. *Nothing could ever have exceeded it.* It read 0.000 at every depth, and that was arithmetic, not evidence.

Probe 2 · the backward pass

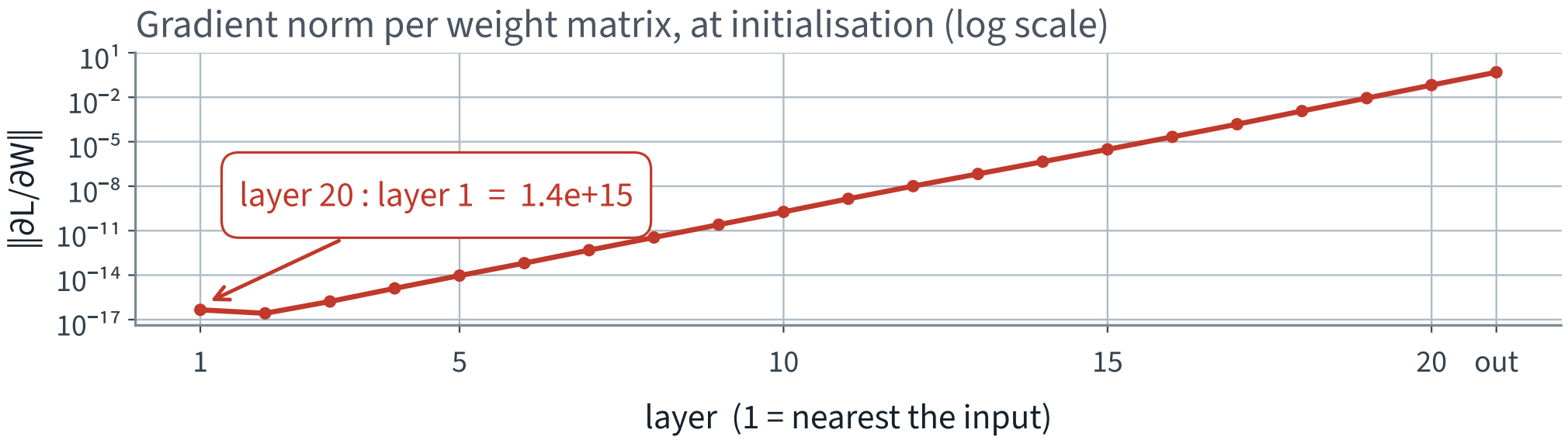
```
lins = [m for m in net if isinstance(m, nn.Linear)]

for _ in range(8):                          # 8 batches, not 1
    net.zero_grad()
    lossf(net(xb), yb).backward()
    acc += np.array([float(m.weight.grad.norm()) for m in lins])

profile = acc / 8
```

Eight batches rather than one, because a single batch of 128 is a noisy estimate of anything and a course that says so should not then quote one.

Gradient norm per weight matrix, at initialisation



✗ Read the axis. Each gridline is a factor of ten, and the line crosses about 15 of them.

The one number to take away

0.1593

The line is **straight** on a log axis, so it is a geometric sequence: not one bad layer, but the *same factor at every layer*. That factor, ρ, is the geometric mean of the ratio between neighbouring layers, over 19 steps.

Per layer, descending	× 0.1593
Over 19 layers	✗ × 6.91e-16
Which is one part in	✗ 1.45e+15

Keep both numbers in view. The mathematics below derives the first from the shapes of the weight matrices, and the second follows.

Does training rescue it?

A fair question: perhaps the first layers are merely slow to start, and the profile flattens as the network learns. We logged the per-layer norms at every epoch, so this is a lookup and not a guess.

Gradient at layer 1	‖∂L/∂W‖
Epoch 1	✗ 6.660e-17
Epoch 20	✗ 4.610e-17

20 epochs of Adam did not move the first layer's gradient into a range where a learning rate of 0.001 could do anything with it.

Why Adam does not rescue this

Adam divides by a running estimate of the gradient's own magnitude, so it is tempting to think a small gradient is automatically compensated.

- It rescales, so the *direction* is what matters — and at 10^{-17} the direction is dominated by batch-to-batch noise
- The ε in the denominator, default 10^{-8}, is larger than the gradients themselves at the bottom of the stack

> An optimiser can compensate for a badly scaled gradient. It cannot manufacture information that the backward pass did not deliver.

> **FAILURE CONDITION — AN OPTIMISER ASKED TO SUPPLY MISSING INFORMATION**
>
> An adaptive optimiser can rescale a badly conditioned gradient. It cannot manufacture information the backward pass never delivered, so a network whose gradient has been attenuated to nothing trains no better under Adam — and the loss curve looks the same either way.

The diagnosis, as far as we can take it

WHAT WE MEASURED

The gradient is attenuated by a constant factor of about 0.159 at every layer, so by the time it reaches the bottom of a 20-layer stack it has been multiplied by 1.4e+15. The layers nearest the data never learn anything.

What we cannot yet say: **why that factor is 0.159 and not 1.**

That is a question about the weight matrices themselves, and it has an exact answer.

The one recursion

One layer, activation dropped: $z_i = \sum_{j=1}^{n_{\text{in}}} w_{ij}\, a_j$, where n_{in} is the fan-in — 100 on our hidden layers, 3,072 on the first. Three assumptions, stated so they can be checked:

1. the weights are drawn independently, with mean zero and variance $\mathrm{Var}(w)$
2. the weights are independent of the inputs — true at initialisation
3. the inputs have mean zero and are uncorrelated with each other

Then each term $w_{ij}a_j$ has mean zero, the terms are uncorrelated, and the variances add:

$$\mathrm{Var}(z) \;=\; n_{\text{in}} \cdot \mathrm{Var}(w) \cdot \mathrm{Var}(a)$$

THE FORWARD VARIANCE RECURSION

The input variance is *multiplied* by a factor at every layer. That is what makes depth dangerous.

Two demands, and Glorot's compromise

To preserve…	the recursion sums over	so it asks for
the forward signal	the fan-in, n_{in}	$\text{Var}(w) = 1/n_{\text{in}}$
the backward gradient, $\bar{a} = W^{\mathsf{T}}\bar{z}$	the fan-out, n_{out}	$\text{Var}(w) = 1/n_{\text{out}}$

The two agree only when $n_{\text{in}} = n_{\text{out}}$ — never on our first layer, 3,072 into 100. So do not choose: take the **harmonic mean** of the two demands.

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

GLOROT (XAVIER) INITIALISATION

A small error in both directions rather than a large one in either. On a square layer it is exact, and reduces to $1/n$.

He’s adjustment

With the activation φ put back, the per-layer factor on the backward signal — the ρ we measured — is $\rho = \sqrt{n_{\text{out}}\,\mathrm{Var}(w)\,\mathbb{E}[\varphi'(z)^2]}$. ReLU’s derivative is 1 half the time and 0 the other half, so $\mathbb{E}[\mathrm{ReLU}'(z)^2] = \frac{1}{2}$: it discards half the variance. Pay for it by doubling the weight variance:

$$\mathrm{Var}(w) = \frac{2}{n_{\text{in}}}$$

HE INITIALISATION, FOR RELU

On our hidden layers, $\mathrm{Var}(w) = 0.0200$ and $\mathrm{sd}(w) = 0.1414$ — against Glorot’s 0.1000 and the default’s 0.0577.

✓ Then $\rho = \sqrt{n_{\text{out}} \cdot \frac{2}{n_{\text{in}}} \cdot \frac{1}{2}} = \sqrt{n_{\text{out}}/n_{\text{in}}}$ — which is 1 exactly when the fans are equal.

Repair 1 · initialisation

```
for m in net:
    if not isinstance(m, nn.Linear):
        continue
    nn.init.xavier_normal_(m.weight)      # Var = 2/(fan_in + fan_out)
    nn.init.zeros_(m.bias)

    # or, for ReLU:
    nn.init.kaiming_normal_(m.weight, nonlinearity="relu")
```

Two lines. `kaiming_normal_` is He; the argument `nonlinearity` is what selects the factor of two, and its default is `"leaky_relu"` with slope 0.01 — near enough, but say what you mean.

Glorot alone, measured

	ρ	End to end	Test accuracy
✗ Default, logistic	✗ 0.1593	✗ 1.4e+15	✗ 10.0%
Glorot, logistic	0.2531	2.2e+11	10.0%

✗ Nearly four orders of magnitude better, and still eleven orders of attenuation — which is hopeless.

Because the logistic's derivative, $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$, is at most $\frac{1}{4}$ for every z. With the weight scale neutralised, ρ can be no larger than that: the measured ρ is now 0.253 — which is 0.25, the logistic's own ceiling. **You cannot initialise your way out of the activation function.**

Repair 2 · the activation function

Every requirement on φ comes from the derivation:

- φ' should reach 1 somewhere, and not be bounded below it
- φ should not saturate on both sides, or gradients die where it does
- Output roughly centred on zero, so assumption 3 holds at the next layer

✓ ReLU satisfies the first two and fails the third. It is still the right default.

ReLU’s own failure mode

DYING UNITS

A unit whose pre-activation is negative for every input in the dataset has gradient exactly zero, forever. It never comes back.

The repairs are all the same idea — give the negative side a non-zero slope:

Variant	Negative side	Cost
Leaky ReLU	$\alpha z, \alpha \approx 0.01$	none
ELU	$\alpha(e^z - 1)$	an exponential per unit
SELU	ELU, scaled to self-normalise	strict conditions on the architecture

A unit that is dead for every input

FAILURE CONDITION

A ReLU whose pre-activation is negative for every row in the dataset has gradient exactly zero forever: it cannot recover, because nothing can move it. The network trains around it and reports a number, with part of its capacity permanently switched off.

Six activations, measured

Activation	Initialisation	Test accuracy
logistic	Glorot	40.4%
tanh	Glorot	40.2%
ReLU	He	37.6%
Leaky ReLU	He	35.4%
✓ ELU	He	**43.3% ✓**
SELU	Glorot	41.8%

Spread: 8.0% points — and the logistic is not last.

All six **with batch normalisation**, which is why the logistic survives: normalisation puts the pre-activations back where its derivative is largest, at every step. Without normalisation, moving off the logistic was worth 30.7% points. With it, the choice of activation is worth 8.0%.

Repair 3 · batch normalisation

Initialisation gets ρ right once. Normalisation makes it right at every step, by standardising each layer's inputs and then learning what scale they should have had.

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \hat{z} + \beta$$

μ_B and σ_B are computed **over the batch**; γ and β are learned. The second step is what stops it from being a straitjacket: the network can undo the normalisation if it needs to.

The bill for batch normalisation

> **ITS STATISTICS DEPEND ON THE OTHER IMAGES IN THE BATCH**
>
> At training time the prediction for one image depends on which images happened to be batched with it. At evaluation time it uses running averages instead. Those are **two different functions**.

Which is Lecture 10's `model.eval()` failure again — and it is now much harder to catch. Dropout wobbles between calls, so running the evaluation twice exposes it. Batch normalisation in training mode is *deterministic* for a fixed batch, so the same trick finds nothing.

✗ One more question to ask: if there is a normalisation layer, was `eval()` called? Running it twice will not tell you.

Normalisation, measured

On the ReLU + He network	Test accuracy	Parameters	Seconds, 3 epochs
✓ No normalisation	40.7% ✓	500,210	5.24
✗ Batch normalisation	✗ 37.6%	504,210	✗ 11.30
✓ Layer normalisation	41.9% ✓	504,210	9.49

✗ Batch normalisation made it worse, by -3.2% accuracy points.

The timing is the *minimum* of 3 repeats: these figures were generated on a shared machine, contention can only make a run slower, so the smallest reading is the closest to the true cost. Batch normalisation costs 2.16× the epoch.

Repair 4 · gradient clipping

```
loss.backward()
torch.nn.utils.clip_grad_norm_(net.parameters(), 1.0)
opt.step()
```

Rescale the whole gradient vector when its norm exceeds a threshold. It preserves the direction and bounds the length.

✗ This is a defence against $\rho > 1$. We have $\rho < 1$. Look at the distribution of norms before choosing a threshold — a clip below the median silently turns your optimiser into something else.

What clipping bought, and what it did not

Gradient clipping at 1.0	Test accuracy
✗ applied alone to the failing network	✗ 10.0% — nothing
✓ added to the repaired network	36.6% → 41.0% ✓

Alone it cannot help: you cannot bound your way out of a gradient that is 1e+15 times too *small*. On the repaired network the step norms have a median of 3.12 under He, so a threshold of 1.0 sits below it and acts as a step-size limiter rather than a safety net, and here that was worth +4.4% points.

✗ Which is a useful effect obtained for the wrong reason. If you want a smaller step, set a smaller learning rate and say so.

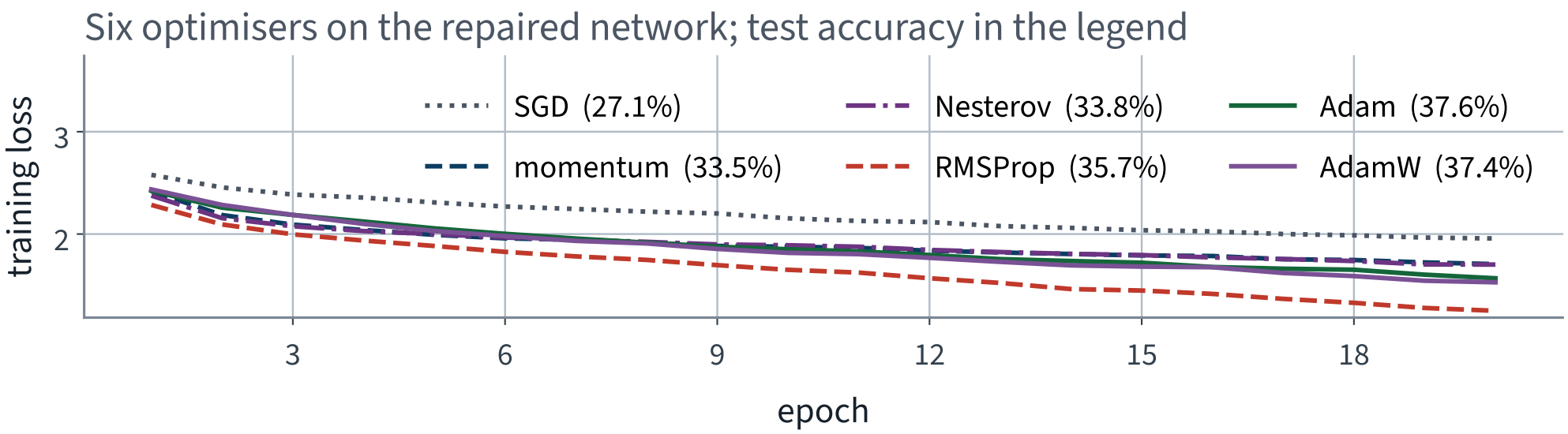
Repair 5 · faster optimisers

Now that the gradient reaches every layer, the optimiser has something to work with. Comparing them on the broken network would have measured nothing.

Optimiser	Idea	Test accuracy
SGD	step along the gradient	27.1%
Momentum	accumulate a velocity	33.5%
Nesterov	measure the gradient ahead of the step	33.8%
RMSProp	divide by a running gradient scale	35.7%
✓ Adam	momentum and RMSProp together	**37.6% ✓**
AdamW	Adam, with the weight decay decoupled	37.4%

SGD, momentum and Nesterov at 0.01; the adaptive three at 0.001 — the same learning rate would not be a fair comparison.

Six optimisers on the repaired network



The spread between the best and the worst optimiser here is far smaller than the effect of the initialisation. Order your effort by the measurement, not by the novelty.

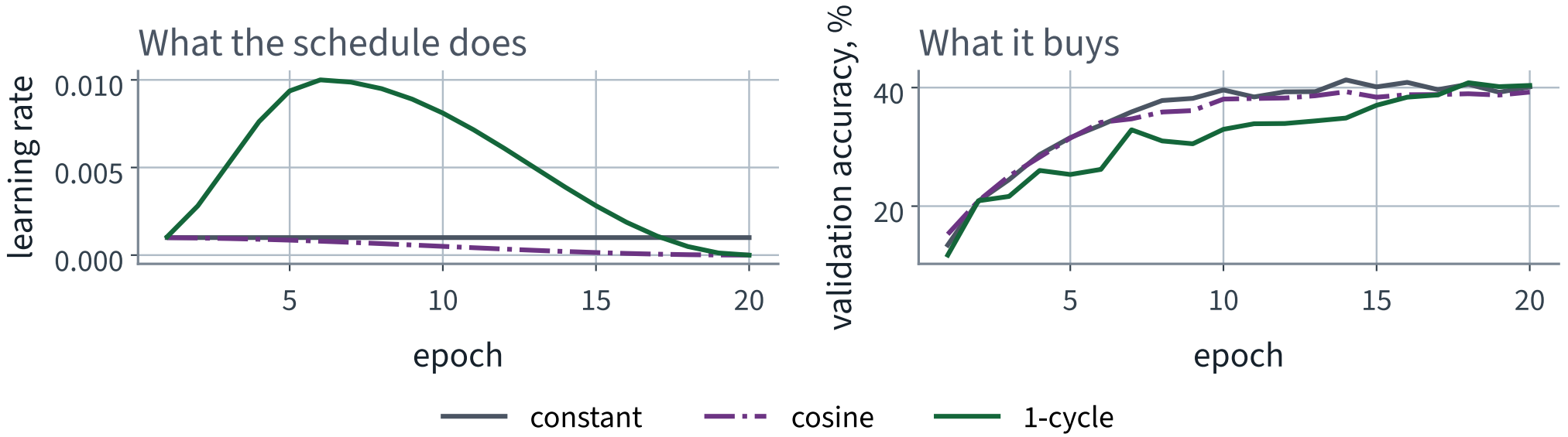
Repair 6 · learning-rate schedules

One step size cannot suit both the start of training and the end. The book gives five schedules; three are worth having.

- **Exponential** — multiply by a constant each epoch
- **Cosine** — anneal smoothly to zero over the run
- **1-cycle** — rise to a peak, then fall well below the starting value

All three need to know how long the run is, which is why they take `total_steps` and why changing the epoch count silently changes the schedule.

What a schedule does, and what it buys



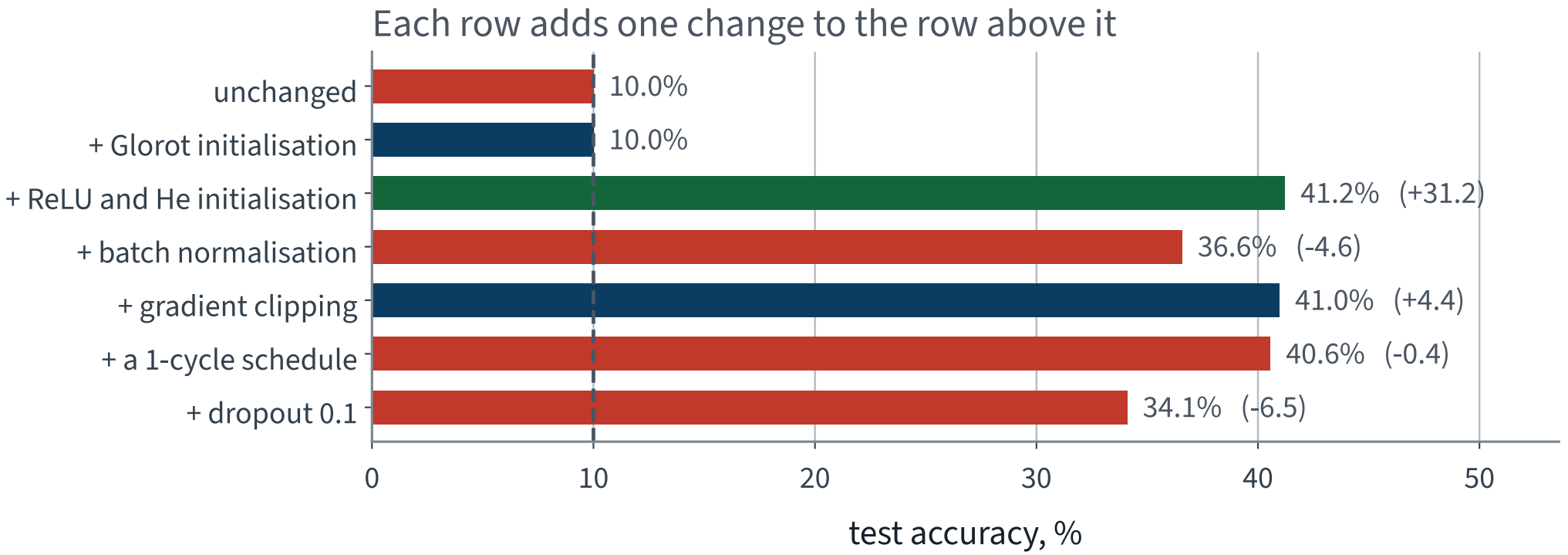
Constant 40.0%, cosine 39.9%, 1-cycle 41.2% — on a twenty-epoch run, one seed each. In the ablation the same schedule is worth -0.4 points against a five-seed spread of ±2.40, so read this panel as a shape, not a ranking.

Each row adds one change to the row above

Configuration	Test accuracy	sd over 5 seeds	Change	Final training loss
✗ the failing network, unchanged	10.0%	±0.00	—	2.3035
+ Glorot initialisation	10.0%	±0.00	+0.0 (noise)	2.3062
✓ + ReLU and He initialisation	41.2%	±0.41	**+31.2 ✓**	1.1324
+ batch normalisation	36.6%	±0.92	**✗ -4.6**	1.5458
✓ + gradient clipping	41.0%	±0.65	**+4.4 ✓**	1.1075
+ a 1-cycle schedule	40.6%	±2.40	-0.4 (noise)	1.2781
+ dropout 0.1	34.1%	±0.62	**✗ -6.5**	1.6252

Five seeds a row; typical spread **0.72 points**. One step is worth **+31.2**, forty times that. **The 1-cycle schedule does not** clear the noise — -0.4 against ±2.40. The last row is dropout, the layer from Lecture 10 that zeroes a random fraction of units in training: a regulariser, added to a network that is not overfitting.

The same table, as a picture



✓ One row moves the number further than the whole journey, and it is the row the mathematics pointed at.

Where the gain came from

	Accuracy points
✓ The largest single step — ReLU with He	**+31.2% ✓**
✓ Best configuration, against the start	**+31.2% ✓**
✗ The worst single step	**✗ -6.5%**
The last row, against the start	+24.1%

One change is worth more than the whole ladder.

Which is the answer to “should I add batch normalisation, or dropout, or a schedule?” — the question is malformed until you know which failure you have. Ours was $\rho \not\checkmark 1$, and only the repairs that address ρ paid.

The silent-failure catalogue so far

Failure	Diagnosed in	Measured cost
Missing `optimizer.zero_grad()`	Lecture 10	66 points
Metric averaged per batch	Lecture 10	0.15 points
✗ A deep stack on default initialisation	**✗ today**	**✗ 31.2% points**

Every one of them ran to completion and printed a plausible number. The last is the first in which no line of code is wrong.

The notebook for this lecture

Open Lecture 11 in Colab

- **Runs on free CPU.** CIFAR-10 is subsampled and the stack is narrow; the point is the *shape* of the gradient curve, which survives the shrink
- The instrumentation: per-layer activation and gradient statistics, logged and plotted
- Each repair applied on its own, so the ablation is something you ran rather than something you read

> **WHAT TO CHANGE**
>
> Set the initialisation back to the default and re-run the gradient probe. The straight line on a log axis comes back, and its slope is the per-layer factor the derivation predicts.

What we did

1. Instrumented a network that would not train, and found the gradients shrinking by a constant factor per layer — a straight line on a log axis
2. Derived why: one layer multiplies the variance by $n \cdot \mathrm{Var}(w)$ — forwards with the fan-in, backwards with the fan-out — so the factor compounds over the depth, and only $\rho = 1$ survives it
3. Saw Glorot's compromise between the two fans, and He's adjustment — doubling $\mathrm{Var}(w)$ because ReLU discards half of it
4. Applied six repairs — initialisation, activation, normalisation, clipping, optimiser, schedule — and measured each
5. Read the ablation, and found that one change, ReLU with He, did more than the whole ladder

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 11

five, in the style of the written paper

Exercises — Lecture 11

1 One layer multiplies the standard deviation of the backward signal by $\rho = \sqrt{n_{\text{out}} \operatorname{Var}(w)\, \mathbb{E}[\varphi'^2]}$. State what L layers do to it, and why only $\rho = 1$ survives depth. [5]

2 Preserving the forward signal asks for $\operatorname{Var}(w) = 1/n_{\text{in}}$ and preserving the gradient asks for $1/n_{\text{out}}$. State when the two agree, and what Glorot does when they do not. [4]

3 He initialisation doubles the weight variance for ReLU. Derive the factor of two in one line. [4]

Solutions on Lecture 12's deck.

Exercises — Lecture 11 (continued)

4 A gradient profile that is a straight line on a log axis tells you something a curved one does not. State what, and why it identifies the cause. [4]

5 Gradient clipping is applied to a network whose gradients are fifteen orders of magnitude too small. Predict the effect, and say what the attempt reveals about the diagnosis. [4]

Solutions on Lecture 12’s deck.

THE FIVE SET AT THE END OF LECTURE 10

Solutions Lecture 10

not part of this lecture

Solutions — Lecture 10

1. Reverse-mode automatic differentiation costs one sweep for all partial derivatives; forward mode costs one…

✓ Many inputs, one output — which is exactly a loss.

- Forward mode's cost scales with the number of inputs, reverse mode's with the number of outputs
- A neural network has millions of parameters and one scalar loss

2. Test accuracy on 10,000 images is computed as the plain mean of the per-batch accuracies, in batches of 384.…

✓ The last batch holds 16 images but is weighted as heavily as a full batch of 384. Count the hits over the whole set, or weight each batch by its size.

- 10,000 = 26 × 384 + 16, so there are 27 batches and the last is short
- Measured, the error was 0.146 points — small here, but you cannot tell from the number whether it was

Solutions — Lecture 10 (2)

3. The five lines of the training loop are reordered so that opt.step() comes before loss.backward(). State what…

✓ Nothing raises, and the training loss stays at its starting level: the parameters are never changed.

- The step is taken with the gradients zeroed at the top of the iteration
- The gradients computed afterwards are cleared before they are ever used

4. A network with dropout is evaluated without calling model.eval(). Describe the symptom precisely, and the…

✓ The score wobbles between identical calls. Evaluate twice and assert the two agree.

- Dropout keeps sampling masks in training mode, so the metric is a random variable
- A deterministic function of fixed weights and fixed data does not change between calls

Solutions — Lecture 10 (3)

5. In the measured comparison the accelerator barely beat the CPU on the smallest network, and its lead grew as…

✓ Dispatch overhead: on a small matrix product much of each step is the cost of dispatching it, not the arithmetic. The crossover is set by how much arithmetic each batch carries — measured, near-parity at the smallest width and nearly four times faster at the widest.

- A GPU is a machine for large matrix products, and a narrow layer at a batch of 128 is not a large one
- The device pulls ahead once each batch carries enough arithmetic to cover the dispatch, and that grows with the layer widths

LECTURE 12 · GÉRON CH. 12

Convolutional networks

Why a flattened image is the wrong representation
Weight sharing and equivariance, derived
Convolution, pooling, and a network built and trained from scratch

Run the Lecture 11 notebook first